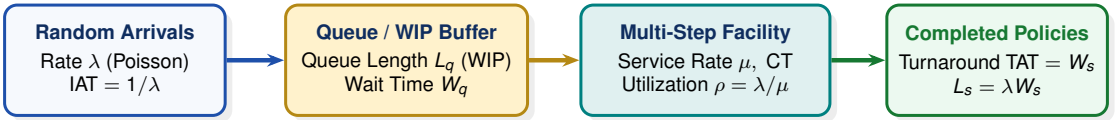
Waiting-Line Models, Capacity Utilization & Service Economics

HBP Simulation Lab, AI Queueing Analytics & Service System Design



"Why does a 33-minute insurance underwriting task take 40 hours to return to the customer—and why do queues explode even when workers are idle? Today we test intuition against queueing physics."

Session Roadmap: 4 Simulation Rounds, 4 Theory Blocks & 5 Gemini Visual Labs

ROUND 1: WIP & TAT
- **Gemini Intuition 1:** Coffee Order Avalanche
- **Sim Round 1 (Cold):** Initial WIP vs. Red TAT
- **Theory 1:** Little's Law $L = \lambda W$ & $\rho < 100\%$

Quiz Q1–Q3

→

ROUND 2: FLOATERS
- **Gemini Intuition 2:** Taco Whack-a-Mole
- **Sim Round 2:** Faster IAT = 7m + Floater
- **Theory 2:** Variability, Poisson λ & Exp. μ

Quiz Q4–Q9

→

ROUND 3: VARIABILITY
- **Gemini Intuition 3:** Robot Wash vs. Salon
- **Sim Round 3:** RUN vs. RAP vs. RERUN
- **Theory 3:** Hockey-Stick & Product-Mix Shifting

Quiz Q10–Q12

→

ROUND 4 & SYNTHESIS
- **Gemini Intuition 4:** Bistro Mix Crossover
- **Sim Round 4:** Mix (Q13)
- **Theory 4:** $M/M/1$, $M/M/S$ Pooling & Maister Laws

13-Question Quiz

Pedagogical Rule: "Visual Intuition First → Live Simulation → Mathematical Theory"

Before each HBP Service Utilization simulation scenario, we build visual intuition with an everyday service analogy and a live Gemini AI chart, then test your decisions in the simulator and formalize the queueing equations.

The HBP Simulation Setting: Manzana-Style Insurance Underwriting

Business Context (Inspired by Manzana Case)

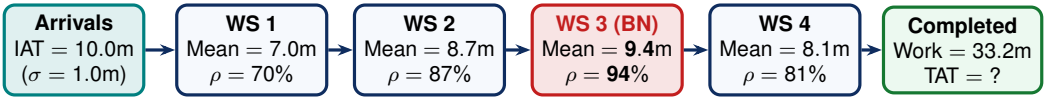
- **Operation:** 4 sequential workstations processing **RUNs** (*Requests for Underwriting*).
- **Horizon:** 12 weeks $\times$ 40 hrs/wk $=$ **480 hrs** $(28,800$ min).
- **Total Direct Labor Content:**

$$7.0 + 8.7 + 9.4 + 8.1 = \textbf{33.2 minutes}$$

- **Strategic Threat:** Competitors promise **same-day turnaround** (≤ 8 hours). Why do our policies take days?

Scenario 1 Baseline Parameters

- **Inter-Arrival Time (IAT):** Mean $=$ **10.0 min** ($\sigma = 1.0$m) $\implies \lambda = 6.0$ policies/hr.
- **Station 3 is the Bottleneck:** $\text{CT}_{\text{BN}} = \textbf{9.4 min} < 10.0$ min ($\rho_{\text{BN}} = 94\%$).
- **Controllable Lever:** Initial Work-in-Process (**WIP**) distributed equally across all 4 stations.



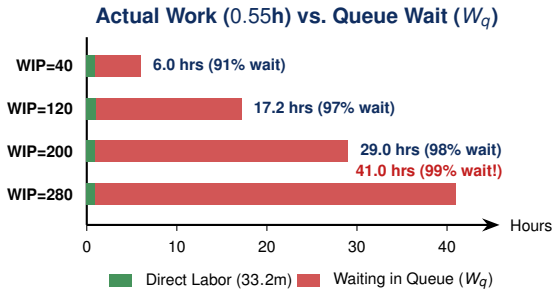
Gemini Visual Intuition 1: “The Coffee Bar Mobile-Order Avalanche”

Everyday Puzzle: Why 4 Min of Work Takes 45 Min

- A 4-step artisanal coffee bar takes only **3.3 min** of barista labor per drink, with the espresso machine as the bottleneck (**0.94** min/cup).
- At 8:00 AM, a mobile-order avalanche dumps **40, 120, 200, or 280 cups** ahead of your ticket.
- **Intuition:** Your wait is governed not by your own drink’s labor, but by WIP Ahead $\times$ Bottleneck Pace!

Copy-Paste Prompt for Gemini AI (Before Scenario 1)

```
A 4-step insurance line has total work = 33.2 min (0.55 hr) and
bottleneck cycle time = 9.4 min.  For Initial WIP = [40, 120,
200, 280], plot a stacked horizontal bar chart comparing Actual
Work Time (0.55 hr) vs Queue Waiting Time so I can see why
Turnaround Time explodes linearly with WIP.
```



FORMULA REMINDER: Turnaround Time (TAT) = Direct Work + Queue Wait $\approx$ WIP $\times$ Bottleneck Cycle Time | More WIP $\implies$ Linearly Longer Wait!

LAB SESSION 1 (COLD RUN): Run Scenario 1 Now! (Questions 1–3)

Step-by-Step Simulation Instructions (Scenario 1 Only)

1. **Run 1 (Initial WIP $= 40$, i.e., 10 per station):**
   - Click **Run** and watch the **Red Policy** enter WS 1 right behind the initial 40 policies.
   - Record the **Turnaround Time (TAT)** of the red policy in hours (**Quiz Q1**).
   - Watch where WIP drains and where it piles up over the 12 weeks.
2. **Run 2 (Sweep Initial WIP):**
   - Test Initial WIP $=$ **120**, **200**, and **280**.
   - Use the linear pattern to estimate TAT at WIP $=$ **240** (**Quiz Q2**, without running 240!).

Record Your Answers for Quiz Q1–Q3

- **Q1:** With WIP $= 40$, what is the Red Policy TAT (in hours, nearest integer)?
- **Q2:** Based on your runs, what would TAT be if WIP $= 240$ (in hours)?
- **Q3 (Steady-State Transfer):** A 15-step process in **steady state** has Average WIP $= 720$ and bottleneck task time $= 5.0$ min. What is Average TAT (in hours)?

STOP AFTER SCENARIO 1! DO NOT OPEN SCENARIO 2!

FORMULA REMINDER: Steady-State Little's Law: Average TAT $=$ Average WIP $\times$ Bottleneck Cycle Time | 1 hour $= 60$ minutes

Scenario 1 Class Discussion: Where Did All That Time Go?

1. The Waiting Paradox

Prompt: Total underwriting work across all 4 workstations is only **33.2 minutes**.

- Why did the Red Policy take **multiple hours (or days)** to exit the facility?
- What percentage of turnaround time was value-added work vs. waiting in queues?

2. Shape of WIP vs. TAT

Prompt: When you increased Initial WIP from 40 $\rightarrow$ 120 $\rightarrow$ 200 $\rightarrow$ 280:

- Was the curve relating Initial WIP to Turnaround Time concave, exponential, or **strictly linear**?
- How many extra minutes of TAT did each additional unit of initial WIP add?

3. Transient Queue Migration

Prompt: At $t = 0$, WIP started equally distributed (25% at each station).

- Why did queues **drain to near zero** at WS 1 (7.0m), WS 2 (8.7m), and WS 4 (8.1m)?
- Why did almost all initial WIP migrate to and **accumulate in front of WS 3** (9.4m)?

Key Diagnostic Question: Why did the slope of TAT/WIP in Scenario 1 slightly exceed WS 3's 9.4 min cycle time? Let's formalize Little's Law and transient WIP shifting.

Theory 1A: Little's Law — Fundamental Conservation Law of Flow ($L = \lambda W$)

Two Equivalent Forms of Little's Law

- **Standard Queueing Form:**

$$\mathbf{L_s = \lambda W_s} \quad \text{and} \quad \mathbf{L_q = \lambda W_q}$$

where L = average items in system/queue, λ = throughput rate, and W = average wait time.

- **HBP Bottleneck Form ($\lambda = 1/\mathbf{CT_{BN}}$):**

$$\text{Average TAT} = \text{Average WIP} \times \text{Bottleneck CT}$$

- **Zero-CapEx Speed Lever:** With fixed capacity, cutting WIP directly slashes customer turnaround time!

Everyday Executive Applications of $L = \lambda W$

- **1. Coffee Bar Queue (W_q):** Arrivals $\lambda = 20$/hr, average line $L_q = 5$ people:

$$W_q = \frac{L_q}{\lambda} = \frac{5}{20/\text{hr}} = 0.25 \text{ hr} = \mathbf{15 \text{ min}}$$

- **2. Retail Occupancy (L_s):** $\lambda = 220$ shoppers/hr spend $W_s = 45$ min (0.75 hr) inside:

$$L_s = \lambda W_s = 220 \times 0.75 = \mathbf{165 \text{ shoppers}}$$

- **3. Pharmacy Shelf Life (W_s):** $L_s = 80$ boxes held, $\lambda = 8$ sold/day $\implies$ $W_s = 80/8 = \mathbf{10 \text{ days}}$.

FORMULA REMINDER: $L_s = \lambda W_s$ | $L_q = \lambda W_q$ | Average TAT = Average WIP × Bottleneck CT | Universal for ANY Steady-State System

Theory 1B: Steady-State Little's Law vs. Transient WIP Shifting

Why Scenario 1 Gave 9.575 $\times$ WIP Instead of 9.4 $\times$ WIP

- **Steady-State Assumption:** In steady state, virtually 100% of WIP sits in front of the bottleneck (WS 3, 9.4 min), so TAT $=$ 9.4 $\times$ WIP.
- **Scenario 1 Transient Start:** The simulation starts **out of steady state** with WIP split equally (25% per station) and tracks **one specific Red Policy** from $t = 0$.
- As the Red Policy moves through WS 1 $\rightarrow$ WS 4, faster upstream stations push work into slower downstream stations!

Exact Trace of the Red Policy at WIP $=$ 120 (30/station)

Station	Policies Processed	Task Time	Elapsed Time
WS 1	30 ahead $+$ 1 red $=$ 31	$\times$7.0 min	217 min
WS 2	38 policies	$\times$8.7 min	331 min
WS 3 (BN)	36 policies	$\times$9.4 min	338 min
WS 4	18 policies	$\times$8.1 min	146 min
Total TAT	**Cumulative Red Policy Time:**		**1,032** min $=$ **17.2** hrs

- While WS 1 takes 217m, WS 2 only finishes $217/8.7 \approx 25$ of its initial 30 policies, leaving $5 + 31 = 36$–38 policies at WS 2!

Theory 1C: Process Capacity Utilization (ρ) & The $\rho < 100\%$ Stability Cliff

Defining Utilization (ρ) in Two Languages

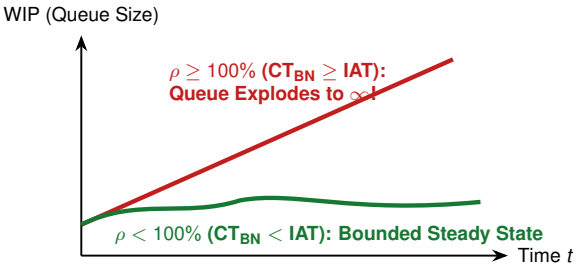
- **Queueing Rate Ratio:**

$$\rho = \frac{\text{Arrival Rate } \lambda}{\text{Bottleneck Service Rate } \mu_{BN}}$$

- **HBP Cycle-Time Ratio:** Since $\lambda = 1/\text{IAT}$ and $\mu_{BN} = 1/\text{CT}_{BN}$:

$$\rho = \frac{\text{Bottleneck Cycle Time } (\text{CT}_{BN})}{\text{Inter-Arrival Time (IAT)}}$$

- **Stability Condition:** A service system reaches steady state **if and only if** $\rho < 100\%$ ($\text{CT}_{BN} < \text{IAT}$).



What Happens When $\rho \geq 100\%$?

Unprocessed work accumulates at rate $(\lambda - \mu_{BN}) > 0$ every hour. WIP and Turnaround Time grow linearly with simulation length toward ∞.

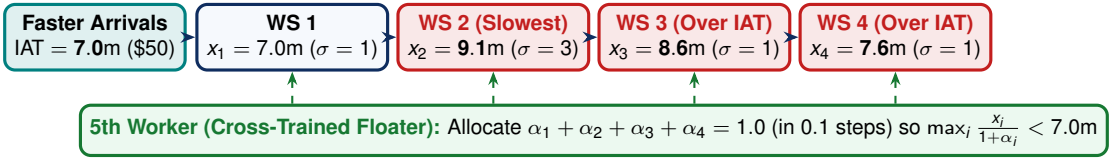
Setting Up Scenario 2: Faster Arrivals (IAT = 7.0 min) & The 5th Worker

Scenario 2 Operating Parameters

- **Faster Demand:** Mean IAT = **7.0 min** ($\sigma = 1.0$m) $\Longrightarrow$ 8.57 policies/hr (4,114 arrivals over 12 weeks).
- **Financials:** Each completed policy pays \$**50** (max potential $\approx$ \$205,700).
- **Initial WIP:** Fixed at 8 policies (2 per station).
- **Total Base Labor:** $7.0 + 9.1 + 8.6 + 7.6 =$ **32.3 min**.

The Trap: 3 Stations Exceed IAT = 7.0 min!

- Without the 5th worker (floater), $t_i(\alpha_i) = \frac{x_i}{1+\alpha_i}$:
  - **WS 1:** 7.0m ($\rho = 100\%$) **WS 2: 9.1m** ($\rho = 130\%$, $\sigma = 3$m)
  - **WS 3: 8.6m** ($\rho = 123\%$) **WS 4: 7.6m** ($\rho = 109\%$)
- Any station left with $t_i \geq 7.0$ min causes the queue to explode!



Gemini Visual Intuition 2: “Bottleneck Whack-a-Mole at the Taco Truck”

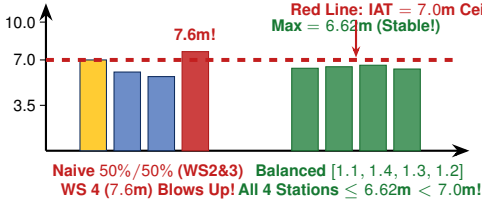
Everyday Puzzle: Why Helping Only the Slowest Fails

- A 4-step Taco Truck (7.0, 9.1, 8.6, 7.6m) receives an order every **7.0** min.
- A 5th floater splits time α_i ($\sum \alpha_i = 1$), cutting station i to $t_i = x_i/(1+\alpha_i)$.
- Helping only Grill & Salsa (50%/50%) leaves **Cashier at** $7.6\mathbf{m} > 7.0\mathbf{m}$!

Copy-Paste Prompt for Gemini AI (Before Scenario 2)

```
Base times x = [7.0, 9.1, 8.6, 7.6]m, IAT = 7.0m.  A 5th worker
is split a = [a1..a4] (sum=1), giving times x_i/(1+a_i).  Plot a
grouped bar chart of station times vs the red dashed 7.0m
ceiling for:  (1) 50/50 WS2&3 vs (2) balanced [0.1, 0.4, 0.3,
0.2].
```

Station Cycle Times (min) vs. IAT $= 7.0$m Ceiling



FORMULA REMINDER: $t_i(\alpha_i) = x_i/(1+\alpha_i)$ | Bottleneck CT $= \max_i t_i(\alpha_i)$ | Full-Balance $\text{CT}_{\text{ideal}} = (x_1+x_2+x_3+x_4)/5 = 32.3/5 = 6.46\text{m} < 7.0\text{m}$

LAB SESSION 2: Run Scenario 2 Now! (Questions 4–9)

Step-by-Step Experiments in Scenario 2

1. **Baseline Bottleneck (Q4):** Before allocating the 5th worker, identify the bottleneck workstation.
2. **100% Floater on Bottleneck (Q5 & Q6):** Assign the entire 5th worker (2.0 workers total) to the initial bottleneck. Record its new task time (**Q5**) and identify which station becomes the **new bottleneck** (**Q6**).
3. **50% / 50% Split (Q7):** Assign 1.5 workers each to the two slowest stations. Run the sim: can the system meet demand ($CT_{BN} < 7.0$m)?
4. **Full Balance & Revenue Maximization (Q8 & Q9):** Calculate the theoretical perfectly balanced cycle time (**Q8**), then test 0.1-increment staffing in the sim to maximize 12-week revenue (**Q9**).

Record Your Answers for Quiz Q4–Q9

- **Q4:** Initial bottleneck station? (*a: WS1, b: WS2, c: WS3, d: WS4*)
- **Q5:** Task time of initial bottleneck with 2 full workers (min)?
- **Q6:** New bottleneck after putting 5th worker on initial bottleneck?
- **Q7:** Does 50%/50% on the two slowest stations meet demand? (*a: No, b: Yes*)
- **Q8:** Lowest possible cycle time if all 4 stations balanced (nearest 0.1m)?
- **Q9:** Max 12-week revenue achieved (nearest $\$1,000$)?

STOP AFTER SCENARIO 2! DO NOT OPEN SCENARIO 3!

FORMULA REMINDER: $t_i(\alpha_i) = \dfrac{x_i}{1+\alpha_i}$ | $CT_{ideal} = \dfrac{7.0+9.1+8.6+7.6}{5} = \dfrac{32.3}{5}$ | Revenue $= \$50 \times$ Completed Policies

Scenario 2 Class Discussion: Breaking the Shifting Bottleneck Trap

1. Why "Fix the Worst Problem" Failed Twice

- **Attempt 1 (100% on WS 2):** Slashed WS 2 from 9.1m $\rightarrow$ 4.55m, but left WS 3 at 8.6m $>$ 7.0m ($\rho = 123\%$). WIP still exploded!
- **Attempt 2 (50%/50% on WS 2 & WS 3):** Reduced WS 2 to 6.07m and WS 3 to 5.73m, but uncovered **WS 4 at** 7.6**m** $>$ 7.0**m** ($\rho = 108.6\%$). WIP still exploded!
- **Lesson:** In tightly coupled lines, adding capacity only to the primary bottleneck fails whenever secondary bottlenecks exceed IAT.

2. The Transition Puzzle to Module 2

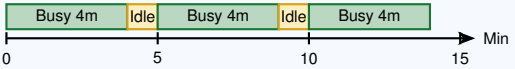
- Once you balanced all 4 stations using 0.1-increment staffing (**1.1**, **1.4**, **1.3**, **1.2** workers), every station had task time $\leq$ 6.62 min $<$ 7.0 min ($\rho \approx 93\%$).
- **Look closely at your balanced WIP chart:** Even with utilization safely below 100%, **why didn't WIP drop to 0?**
- Why did WIP continually bounce up and down between 4 and 8 policies throughout the 12 weeks?

Next Big Question: If a facility has more capacity than average demand ($\rho <$ 100%), why do queues ever form at all? Answer: **Arrival & Service Variability**.

Theory 2A: The Variability Puzzle — Why Queues Form When $\rho < 100\%$

1. Deterministic Benchmark ($\sigma_a = 0,\ \sigma_s = 0$)

- **Setup:** $\lambda = 12$ cust/hr (IAT = exact 5 min) and service takes an exact 4 min ($\mu = 15$/hr).
- **Utilization:** $\rho = 12/15 = \mathbf{80\%}$ (server idle 1 min every 5 min).
- **Wait Time:** $\mathbf{W_q = 0}$ **min**, $\mathbf{L_q = 0}$. Zero queue ever forms!



2. Arrival Bunching Thought Experiment (Same $\rho = 80\%$!)

- Suppose the **same 12 customers/hr** arrive together in a clump at $t = 0$, and each still takes 4 min:
  - Customer 1 waits **0** min; Customer 2 waits **4** min; ...
  - Customer 12 waits $11 \times 4 = \mathbf{44}$ **min**!
  - **Average Wait:** $W_q = \frac{0+4+8+\cdots+44}{12} = \mathbf{22.0}$ **min**!
- **Core Insight:** Queues when $\rho < 100\%$ are caused by **random bunching of arrivals** and **service time variability**—idle time cannot be "saved up" in a bank!

Theory 2B: Why We Model Random Arrivals as "Poisson" (λ)

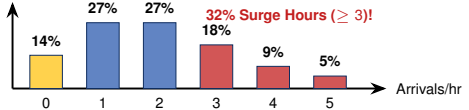
What Does "Poisson Arrivals" Mean in Plain English?

- **1. Unscheduled & Independent:** Walk-in patients, callers, or insurance submissions decide to arrive *independently* of one another.
- **2. Random Bursts & Gaps ("Feast or Famine"):** Even when the average arrival rate is calm ($\lambda = 2$ cars/hr), arrivals do **not** space themselves 30 minutes apart!
- **3. Temporary Surge Overloads:** Some hours see 0 arrivals (wasting server idle time), while other hours see sudden bursts of 4 or 5 arrivals that overwhelm the server!

1. Clockwork vs. Random Poisson Arrival Stream:



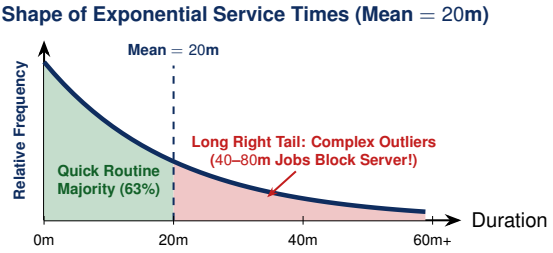
2. How Often Do Surge Hours Happen at $\lambda = 2$/hr?



Theory 2C: Why We Model Random Service Times as “Exponential” (μ)

Three Operational Realities of Service Work

- **1. Most Routine Jobs Are Quick:** In tech support, ER triage, or underwriting, the majority of routine requests finish faster than the average time ($1/\mu$).
- **2. Complex Outliers Block the Line:** A small fraction of “nightmare” cases take **3** $\times$ **to 4**$\times$ the average duration, tying up the server while new arrivals pile up behind them!
- **3. High Natural Variability (CV** $= 1$**):** Under an Exponential service distribution, the **standard deviation equals the mean** ($\sigma_s =$ Mean).



The Classic $M/M/1$ Benchmark: A Single-Bay Muffler Repair Shop

Arnold's Muffler Shop Operating Data

- **Arrivals (Poisson):** $\lambda = 2$ **cars/hr**
  $\Longrightarrow$ Average Inter-Arrival Time IAT $= 30$ **minutes**.
- **Service (Exponential):** $\mu = 3$ **cars/hr**
  $\Longrightarrow$ Average Service Time $1/\mu = 20$ **minutes**.
- **Mechanic Utilization (ρ):**

$$\rho = \frac{\lambda}{\mu} = \frac{2}{3} = 66.7\%$$

- **Idle Time:** $P_0 = 1 - \rho = 33.3\%$ (the mechanic sits idle **2.67 hours** out of every 8-hour shift!).

Intuition Challenge Before We Simulate

Question for the Class:

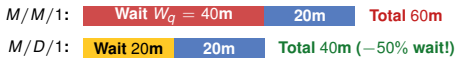
- Cars arrive every **30 minutes** on average, and the mechanic finishes a car in **20 minutes** on average (working only 66.7% of the time).
- **How long does the average customer wait in line (W_q) before their car even enters the bay?**
  1. About 0–5 minutes?
  2. About 10–15 minutes?
  3. **Longer than the 20-minute repair itself?**

Instead of building a 1,000-row Excel spreadsheet (`A Repair Shop Model MM1.xlsx`), let's use **Gemini AI** to simulate 10,000 cars in 10 seconds!

Live Gemini AI Exercise: Monte Carlo Repair Shop ($M/M/1$ vs. $M/D/1$)

Copy-Paste Prompt for Gemini AI (Replaces Excel Lab)

```
Simulate 10,000 cars arriving at a single-bay shop with Poisson
arrivals lambda = 2 cars/hr (IAT = 30m) and mean service rate mu
= 3 cars/hr (service = 20m).  Compare:
1) M/M/1:  Exponential service (mean 20m, sd 20m).
2) M/D/1:  Constant service (exact 20m, sd 0m).
Plot a side-by-side stacked bar chart of Queue Wait (Wq) +
Service Time (20m) and verify with exact M/M/1 and M/D/1
formulas.
```



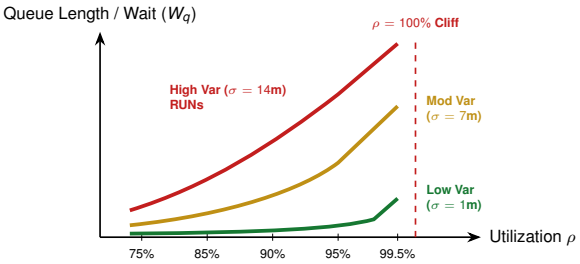
The Shocking Result ($\rho = 66.7\%$ in Both!)

Metric	**Case 1:** $M/M/1$ *(Exp. 20m)*	**Case 2:** $M/D/1$ *(Const. 20m)*
Utilization ρ	66.7%	66.7%
Idle Fraction P_0	33.3%	33.3%
Avg Queue L_q	**1.33** cars	**0.67** cars
Avg Wait W_q	**40.0 min**	**20.0 min**
Total Time W_s	**60.0 min**	**40.0 min**

- Even with a mechanic idle 1/3 of the day, an $M/M/1$ customer waits **40 minutes** in line (2× the repair time!).
- Standardizing service time ($M/D/1$) **cuts wait time in half (40m → 20m)** without hiring anyone!

FORMULA REMINDER: $M/M/1$: $L_q = \frac{\lambda^2}{\mu(\mu-\lambda)}$, $W_q = \frac{\lambda}{\mu(\mu-\lambda)}$ | $M/D/1$: $L_q = \frac{\lambda^2}{2\mu(\mu-\lambda)}$, $W_q = \frac{\lambda}{2\mu(\mu-\lambda)}$ | $W_s = W_q + \frac{1}{\mu}$

Theory 2D: The Utilization $\times$ Variability "Hockey-Stick" Curve



Why Utilization & Variability Multiply

- **Kingman's Queueing Approximation:**

$$\mathbf{W_q} \approx \left(\frac{\mathbf{1}}{\mu}\right) \cdot \underbrace{\left(\frac{\rho}{1-\rho}\right)}_{\text{Utilization}} \cdot \underbrace{\left(\frac{\mathrm{CV}_a^2 + \mathrm{CV}_s^2}{2}\right)}_{\text{Variability}}$$

- **The Nonlinear $\frac{\rho}{1-\rho}$ Multiplier:** Equals **4** at $\rho = 80\%$, **9** at $\rho = 90\%$, **19** at $\rho = 95\%$, and **199** at $\rho = 99.5\%$!
- **Target Utilization Ceilings:**
  - **Low Var ($\sigma = 1$m):** Safe up to **95%**+ utilization.
  - **Mod Var ($\sigma = 7$m):** Keep $\rho <$ **90%** (10% buffer).
  - **High Var ($\sigma = 14$m):** Keep $\rho <$ **75%–80%** (20%+ buffer!).

FORMULA REMINDER: $W_q \propto \left(\frac{\rho}{1-\rho}\right) \times$ Variability | High ρ + High σ = Queue Explosion | More Variance Requires Larger Capacity Buffer $(1-\rho)$

Setting Up Scenario 3: Three Policy Types with Different Variances

Scenario 3 Product Portfolio (IAT $= 20.0$ min, $\sigma_a = 1.0$m)

All 4 workstations have the **exact same mean task time** t ($17, 18, 19, 19.9,$ or 20.9 min), but the three policy types differ dramatically in **task-time standard deviation (σ)**:

Policy Type	Description	Task Std Dev (σ)	Revenue
RUN	New Underwriting	**14.0 min** *(High)*	\$50
RAP	Price Approval	**7.0 min** *(Moderate)*	\$42
RERUN	Standard Renewal	**1.0 min** *(Low)*	\$30

Selectable Utilization Levels (IAT $= 20.0$m)

Task Time t	Utilization $\rho = t/20$	Regime
17.0 min	85.0%	Comfortable Buffer
18.0 min	90.0%	Moderate Load
19.0 min	95.0%	High Load
19.9 min	**99.5%**	Razor's Edge
20.9 min	**104.5%**	**Unstable ($\rho > 100\%$)**

Experimental Goal: Hold the mean task time identical across RUN, RAP, and RERUN to isolate the pure operational impact of **standard deviation (σ)** and **utilization (ρ)** on WIP and Turnaround Time.

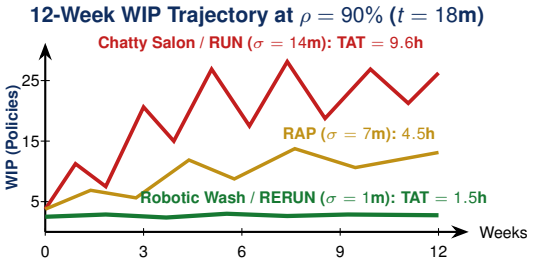
Gemini Visual Intuition 3: “Robotic Car Wash vs. Chatty Hair Salon”

Everyday Puzzle: Same 90% Utilization, 6× Wait!

- Both businesses receive a customer every **20** min and take an average of **18** min per job ($\rho = \mathbf{90}\%$):
  - **Robotic Laser Wash (RERUN, $\sigma = 1$m):** Every wash takes 17–19 min like clockwork $\implies$ tiny line!
  - **Chatty Hair Salon (RUN, $\sigma = 14$m):** Some trims take 5m, but a 45m perm triggers a massive backlog that takes days to clear at 90% load!

Copy-Paste Prompt for Gemini AI (Before Scenario 3)

```
Simulate 1,440 arrivals (IAT = 20m, sd = 1m) at a station with
mean service time = 18m (90% utilization).  Plot the queue
length over 12 weeks on the same chart for:  (1) Robotic Wash
(sd = 1m), (2) Mid-Var (sd = 7m), and (3) Chatty Salon (sd =
14m).  Also plot Average TAT at t = [18, 19, 19.9] min.
```



FORMULA REMINDER: $\rho = \dfrac{t}{\text{IAT}} = \dfrac{18\text{m}}{20\text{m}} = 90\%$ | Same Mean $t = 18$m, But $\sigma = 14$m Has 6.4 × Longer TAT Than $\sigma = 1$m! | $W_q \propto \dfrac{\rho}{1-\rho} \cdot \sigma^2$

LAB SESSION 3: Run Scenario 3 Now! (Questions 10–12)

Step-by-Step Experiments in Scenario 3

1. **Experiment A — Pure Variability Effect (Q10):**
   - Set task time $t =$ **18.0 min** ($\rho = 90\%$) at all 4 stations.
   - Test (i) 100% RERUNs ($\sigma = 1$), (ii) 100% RAPs ($\sigma = 7$), and (iii) 100% RUNs ($\sigma = 14$). Observe Average TAT (**Q10**).
2. **Experiment B — The Hockey-Stick Jump (Q11 & Q12):**
   - Set mix to 100% RUNs ($\sigma = 14$m). Run at $t =$ **18m**, **19m**, and **19.9m**.
   - Compare the TAT jump from 18 $\rightarrow$ 19m vs. 19 $\rightarrow$ 19.9m (**Q11, Q12**).
3. **Experiment C — Over-Capacity Test:**
   - Test $t =$ **20.9 min** ($\rho = 104.5\%$) on 100% low-variance RERUNs ($\sigma = 1$). Does low variance save an overloaded system?

Record Your Answers for Quiz Q10–Q12

- **Q10:** At $t = 18$m, as variability (σ) increases, Average TAT: (*a: increases, b: decreases, c: stays same*)
- **Q11:** For 100% RUNs, as mean task time increases (18 $\rightarrow$ 19 $\rightarrow$ 19.9m), Average TAT: (*a: increases, b: decreases, c: stays same*)
- **Q12:** Which step causes the **larger** jump in Average TAT? (*a:* 18 *to* 19 *min, b:* 19 *to* 19.9 *min*)

STOP AFTER SCENARIO 3! DO NOT OPEN SCENARIO 4!

FORMULA REMINDER: $\rho = \dfrac{\text{Task Time } t}{\text{IAT (20m)}}$ | $W_q \propto \left(\dfrac{\rho}{1-\rho}\right) \times (\text{CV}_a^2 + \text{CV}_s^2)$ | At $\rho \rightarrow 100\%$, $\dfrac{\rho}{1-\rho} \rightarrow \infty$

Scenario 3 Class Discussion: Why 99.5% Utilization Is a Trap

1. Same Mean, 6× Wait!

Prompt: At $t = 18$ min ($\rho = 90\%$), every workstation has 10% idle capacity regardless of policy type.

- Why did 100% RERUNs ($\sigma = 1$m) sail through in ~ 1.5 hours while 100% RUNs ($\sigma = 14$m) took ~ 9.6 hours?

2. The Nonlinear Cliff

Prompt: Increasing task time from $18 \rightarrow 19$ min adds 1.0 min of work, while $19 \rightarrow 19.9$ min adds only 0.9 min of work.

- Why did the smaller 0.9 min increase ($19 \rightarrow 19.9$m) produce a **much larger explosion** in WIP and Turnaround Time?

3. When $\rho > 100\%$ (20.9m)

Prompt: In Experiment C, you ran low-variance RERUNs ($\sigma = 1$m) at $t = 20.9$ min ($\rho = 104.5\%$).

- Why did WIP still climb relentlessly to 81 policies?
- Can Six Sigma variance reduction ever rescue a process with $\rho > 100\%$? (**Never!**)

Executive Takeaway: Traditional accounting views 99.5% worker utilization as "peak efficiency." Operations science proves that in high-variance services, 99.5% utilization is **self-inflicted gridlock**.

Theory 3A: Multi-Product Operations & Shifting Bottlenecks

Why Real-World Service Lines Rarely Have a Fixed Bottleneck

- In Scenarios 1–3, a single station remained the bottleneck because all jobs had identical relative work profiles.
- **Real Service Operations** (hospitals, insurance underwriting, commercial banks, semiconductor fabs) process a **heterogeneous product mix**:
  - **Product A** may require heavy legal/actuarial review at **Workstation 3**.
  - **Product B** may require heavy pricing/credit checks at **Workstation 2**.

The Product-Mix Paradox

- What if **100% Product A** is unsustainable because its bottleneck exceeds IAT ($CT_{BN}^{(A)} > IAT$)...
- **AND 100% Product B** is *also* unsustainable because its bottleneck exceeds IAT ($CT_{BN}^{(B)} > IAT$)?
- **Counterintuitive Truth:** If Product A and Product B have **complementary bottlenecks** (peaking at *different* workstations), a strategic mix of A and B can make the entire factory **stable and highly profitable**!

Theory 3B: The Mathematics of Complementary Bottlenecks

Weighted-Average Station Cycle Time

- Suppose a facility processes fraction α of Product A and fraction $(1-\alpha)$ of Product B.
- The effective average cycle time at Workstation j ($j = 1, 2, 3, 4$) is the convex combination:

$$\mathrm{CT}_{\mathbf{j}}(\alpha) = \alpha\, \mathbf{t}_{\mathbf{A},\mathbf{j}} + (\mathbf{1} - \alpha)\, \mathbf{t}_{\mathbf{B},\mathbf{j}}$$

- **Multi-Product Stability Condition:** Every workstation j must have an effective cycle time strictly below Inter-Arrival Time (IAT):

$$\max_{\mathbf{j} \in \{\mathbf{1},\mathbf{2},\mathbf{3},\mathbf{4}\}} \mathrm{CT}_{\mathbf{j}}(\alpha) < \mathrm{IAT}$$

Same Bottleneck vs. Complementary Bottlenecks

- **Case 1: Same Bottleneck Station (Fails!)**
  - If both products peak at WS 3 with $t_{A,3} = 9.4$m and $t_{B,3} = 9.5$m while IAT $= 9.0$m, then for *any* $\alpha \in [0, 1]$, $\mathrm{CT}_3(\alpha) \in [9.4, 9.5] > 9.0$m.
- **Case 2: Complementary Bottlenecks (Succeeds!)**
  - Product A peaks at **WS 3** (9.4m) but is fast at **WS 2** (8.7m).
  - Product B peaks at **WS 2** (9.4m) but is fast at **WS 3** (7.8m).
  - Mixing A and B pulls **both** WS 2 and WS 3 below 9.0 min!

Setting Up Scenario 4: Multi-Product Task Matrix & Revenue Data

Scenario 4 Task Matrix (IAT = **9.0 min**, $\sigma_a = 1.0$m)

Over 12 weeks (480 hrs $= 28,800$ min), exactly **3**, **200 policies** arrive. Look closely at each policy type's bottleneck (highlighted in **red**):

Policy	Rev	Std Dev	WS 1	WS 2	WS 3	WS 4
RUN	$**50**	$\sigma = 5.0$m	7.0m	8.7m	**9.4**m	8.1m
RERUN	$30	$\sigma = 0.2$m	7.8m	7.1m	**9.5**m	8.8m
RAP	$**42**	$\sigma = 2.5$m	7.4m	**9.4**m	7.8m	8.6m

Three Crucial Observations

1. **Every pure (**100%**) product is unstable!** Each has a bottleneck of 9.4m or 9.5m $>$ IAT (9.0m).
2. **RERUNs are a trap:** Not only do they pay the lowest revenue ($30), they share the **exact same WS 3 bottleneck (**9.5**m)** as RUNs (9.4m)!
3. **RUNs & RAPs are complementary:** RUN peaks at WS 3 (9.4m), while RAP peaks at WS 2 (9.4m).

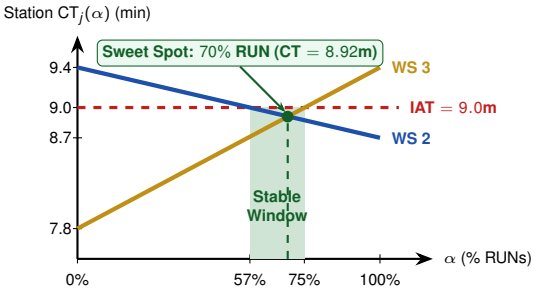
Gemini Visual Intuition 4: “The Two-Dish Bistro Crossover”

Everyday Puzzle: Balancing Grill vs. Sauté Station

- A bistro receives an order every **9.0** min:
  - **Steak (RUN,** \$50**):** Fast on Sauté (WS 2 $= 8.7$m), overloads Grill (WS 3 $= \mathbf{9.4}$m > 9.0m).
  - **Pasta (RAP,** \$42**):** Overloads Sauté (WS 2 $= \mathbf{9.4}$m > 9.0m), fast on Grill (WS 3 $= 7.8$m).
- Neither 100% Steak nor 100% Pasta works—but mixing them crosses the two workload lines **below** 9.0 min!

Copy-Paste Prompt for Gemini AI (Before Scenario 4)

```
Plot WS2 cycle time CT2(a) = 8.7*a + 9.4*(1-a) and WS3 cycle
time CT3(a) = 9.4*a + 7.8*(1-a) for RUN share a in [0, 1]
against the red dashed IAT = 9.0m ceiling.  Shade the
Goldilocks Stability Window (57.1% < a < 75.0%) where both
lines are below 9.0m, and mark the crossing point at a = 70%.
```



FORMULA REMINDER: $CT_2(\alpha) = 9.4 - 0.7\alpha < 9.0 \implies \alpha > 57.1\%$ | $CT_3(\alpha) = 7.8 + 1.6\alpha < 9.0 \implies \alpha < 75.0\%$ | Crossing Point: $\alpha = 69.6\% \approx 70\%$ RUN

LAB SESSION 4: Run Scenario 4 Now! (Question 13)

Step-by-Step Experiments in Scenario 4

1. **Test 1 — Verify Single-Product Instability:**
   - Run 100% RUNs and observe WS 3 queue explosion and total revenue.
   - Try mixing RUNs and low-variance RERUNs (e.g., 50%/50%). Confirm why WS 3 still explodes!
2. **Test 2 — Complementary RUN / RAP Mixes:**
   - Set RERUN $= 0\%$. Test mixes inside the feasible stability window $\alpha \in (57.1\%,\ 75.0\%)$ of RUNs and $(1-\alpha)$ of RAPs.
   - Compare the boundary mix (**75% RUN/25% RAP**, where WS 3 hits $CT_3 = 9.0\text{m},\ \rho = 100\%$) against the balanced mix (**70% RUN/30% RAP**, where $CT_2 \approx CT_3 = 8.92\text{m},\ \rho = 99.1\%$).

Record Your Answer for Quiz Q13

- **Q13:** What is the **highest 12-week total revenue** you can achieve in Scenario 4 (rounded to the nearest $\$1,000$)?
- **Check Your Screen:** Look at the WIP graph for 70% RUN / 30% RAP vs. 75% RUN / 25% RAP:
  - Why does 75% RUN leave dozens of high-value policies stranded in WIP at week 12?
  - How much revenue is lost in uncompleted WIP when $\rho = 100\%$?

FORMULA REMINDER: Feasible Window: $57.1\% < \alpha_{\text{RUN}} < 75.0\%$ | Balanced Mix: $\alpha_{\text{RUN}} \approx 70\%,\ \alpha_{\text{RAP}} \approx 30\%$ | $\text{Rev} = \$50N_{\text{RUN}} + \$42N_{\text{RAP}} + \$30N_{\text{RERUN}}$

Scenario 4 Class Discussion: Why 70% RUN / 30% RAP Wins

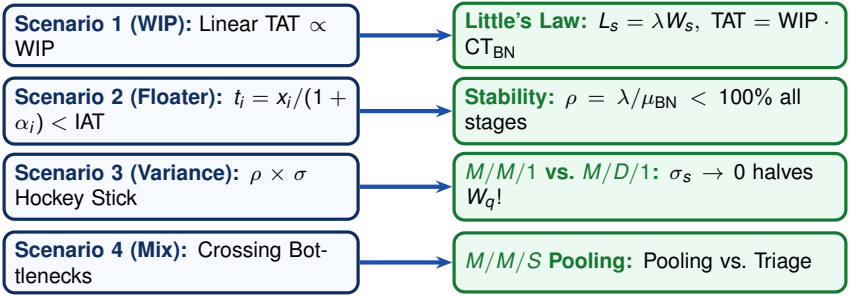
1. Why Low-Variance RERUNs Couldn't Help

- Students often try adding RERUNs because $\sigma_{\text{RERUN}} = 0.2$ min is tiny (remembering Scenario 3!).
- **Why it fails here:** Variance reduction only helps when $\rho < 100\%$. Because both RUN (9.4m) and RERUN (9.5m) overload WS 3 (IAT $= 9.0$m), any mix of RUN and RERUN has $\rho_{\text{WS3}} \geq 104.4\% > 100\%$!
- **Hierarchy of Operations:** First achieve $\rho < 100\%$ via complementary bottlenecks; *then* manage variability.

2. Greedy 75% RUN ($\rho = 100\%$) vs. Balanced 70% RUN ($\rho = 99.1\%$)

- On paper, 75% RUN / 25% RAP has a higher average price per arrival ($48.00) than 70% RUN / 30% RAP ($47.60).
- **In the simulation:** At 75% RUN, $\text{CT}_3 = 7.8 + 1.6(0.75) = \mathbf{9.00\ min} \implies \rho_{\text{WS3}} = \mathbf{100\%}$!
- Combined with RUN's high variance ($\sigma = 5.0$m), running WS 3 at 100% utilization causes massive WIP build-up, stranding uncompleted policies inside the building when the 12 weeks end!

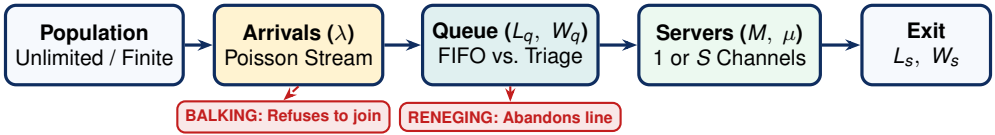
Synthesizing the 4 Simulation Scenarios into Formal Queueing Theory



What Remains to Complete Your Executive Toolkit?

In Scenarios 1–4, each station acted as a serial flow step. Now we formalize **single-server vs. multi-server parallel pooling ($M/M/1$, $M/M/S$, $M/D/1$), waiting-cost vs. capacity-cost optimization**, and **the psychology of waiting lines**.

Anatomy of a Waiting-Line System & Customer Behavior



Queue Discipline Rules

- **FIFO (First-In, First-Out):** Default fairness rule in retail, banking, restaurants.
- **Priority / Triage Discipline:** Emergency rooms (critical trauma first), TSA PreCheck, or Shortest-Processing-Time (SPT) express lanes.

The 7 Standard Queueing Metrics

- $\mathbf{L_q}$, $\mathbf{W_q}$: Average number & wait time **in queue**.
- $\mathbf{L_s}$, $\mathbf{W_s}$: Average number & total time **in system**.
- ρ, $\mathbf{P_0}$, $\mathbf{P_{n>k}}$: Utilization, idle probability, and tail risk of $> k$ units in system.

The Executive Queueing Toolkit: Models A, B, and C

Model (Kendall)	Channels	Arrival & Service	Core Equations ($\rho < 1$)	Service Examples
Model A: $M/M/1$ ***(Single Server)***	$M = 1$ server Single phase	Poisson arrivals (λ) Exp. service (μ)	$L_s = \lambda/(\mu - \lambda)$, $W_s = 1/(\mu - \lambda)$ $L_q = \lambda^2/[\mu(\mu - \lambda)]$, $W_q = L_q/\lambda$	Single-bay repair shop, solo clinic desk
Model B: $M/M/S$ ***(Multi-Server Pool)***	$M \geq 2$ servers 1 snake line	Poisson arrivals (λ) Exp. service (μ/server)	$\rho = \lambda/(M\mu) < 1$, $L_s = L_q + \lambda/\mu$ $W_q = L_q/\lambda$, $W_s = W_q + 1/\mu$	Airline check-in, bank snake line, call center
Model C: $M/D/1$ ***(Constant Service)***	$M = 1$ server Single phase	Poisson arrivals (λ) **Constant** ($\sigma_s = 0$)	$L_q = \lambda^2/[2\mu(\mu-\lambda)] = 0.5\, L_q^{(M/M/1)}$ $W_q = \lambda/[2\mu(\mu - \lambda)] =$ $0.5\, W_q^{(M/M/1)}$	Automated car wash, robotic compactor

Universal Linkage Across All Three Models (Little's Law + Service Time)

For **every** model above, once you know L_q (by formula, table, or Gemini AI), the remaining three metrics follow immediately via:

$$\mathbf{W_q} = \frac{\mathbf{L_q}}{\lambda}, \qquad \mathbf{W_s} = \mathbf{W_q} + \frac{\mathbf{1}}{\mu}, \qquad \mathbf{L_s} = \lambda \mathbf{W_s} = \mathbf{L_q} + \frac{\lambda}{\mu}$$

Model A ($M/M/1$) Walkthrough & Congestion Tail Risk ($P_{n>k}$)

Arnold's Muffler Shop ($\lambda = 2$/hr, $\mu = 3$/hr)

- $L_s = \dfrac{\lambda}{\mu - \lambda} = \dfrac{2}{3-2} =$ **2.0 cars in system**
- $W_s = \dfrac{1}{\mu - \lambda} = \dfrac{1}{3-2} =$ **1.0 hr = 60 min total**
- $L_q = \dfrac{\lambda^2}{\mu(\mu - \lambda)} = \dfrac{2^2}{3(3-2)} =$ **1.33 cars waiting**
- $W_q = \dfrac{\lambda}{\mu(\mu - \lambda)} = \dfrac{2}{3(1)} = \dfrac{2}{3}$ hr = **40 min wait**
- $\rho = \dfrac{\lambda}{\mu} =$ **66.7% busy**, $P_0 = 1 - \rho =$ **33.3% idle**

Parking Lot Sizing: Tail Risk $P_{n>k} = (\lambda/\mu)^{k+1}$

What is the probability that **more than** k **cars** are in the shop at the same time?

k	**Formula** $(2/3)^{k+1}$	**P(n > k)**
0	$(2/3)^1$	66.7% *(Mechanic busy)*
1	$(2/3)^2$	44.4% *($\geq$ 1 car waiting)*
2	$(2/3)^3$	29.6%
3	$\mathbf{(2/3)^4}$	**19.8**% *(> 3 cars in shop!)*
4	$(2/3)^5$	13.2%
6	$(2/3)^7$	5.8%

Single-Channel Capacity Economics: Waiting Cost vs. Service Cost

Total Daily Queueing Cost Framework

- Operations managers must minimize the sum of **Service Capacity Cost** and **Customer Waiting Cost**:

$$\text{Total Cost} = \mathbf{M} \cdot \mathbf{C}_{\text{server}} + \text{Wait Hrs} \cdot \mathbf{C}_{\text{wait}}$$

- Notice that **Daily Customer Wait Hours** over an 8-hour day can be computed two equivalent ways:

$$\text{Daily Wait Hrs} = (8\lambda) \times W_q = 8 \times L_q$$

Arnold's Baseline Cost ($M = 1,\ \mu = 3$/hr)

- **Parameters (8-Hour Workday):**
  - Daily arrivals: 8 hrs $\times$ 2 cars/hr = **16 cars/day**
  - Mechanic wage (C_{server}): \$11/hr $\times$ 8 hrs = **\$88/day**
  - Customer goodwill / wait cost (C_{wait}): **\$15/hr**
- **Baseline Cost Calculation ($W_q = \frac{2}{3}$ hr):**

$$\text{Wait Hrs/Day} = 16 \times \frac{2}{3}\ \text{hr} = \mathbf{10.67\ hrs/day}$$

$$\text{Daily Wait Cost} = 10.67 \times \$15 = \mathbf{\$160/day}$$

$$\text{Total Daily Cost} = \$88 + \$160 = \mathbf{\$248/day}$$

Model B ($M/M/S$): The Nonlinear Magic of Adding a Second Bay

What Happens If Arnold Opens a 2nd Bay ($M = 2$)?

Same arrivals ($\lambda = 2$/hr), same mechanic speed ($\mu = 3$/hr each). Look at the **16-fold collapse** in waiting time:

Operating Metric	**M = 1** Bay	**M = 2** Bays	**Impact**
Utilization $\rho = \frac{\lambda}{M\mu}$	66.7%	33.3%	Halved
System Empty P_0	0.333	0.500	+50%
Cars in System L_s	2.00 cars	0.75 cars	−62.5%
Time in System W_s	60.0 min	22.5 min	−62.5%
Queue Length L_q	**1.333** cars	**0.083** cars	**16** × Drop!
Wait in Queue W_q	**40.0** min	**2.5** min	**16** × Drop!

P&L Proof: Why Hiring Mechanic #2 Pays for Itself!

- **Wait Time with $M = 2$ Bays ($W_q = 2.5\mathbf{m} = \frac{1}{24}$ hr):**

$$16 \text{ cars} \times \frac{1}{24} \text{ hr} = \mathbf{0.67 \text{ wait-hrs/day}}$$

- **New Daily Waiting Cost:**

$$0.67 \text{ hrs} \times \$15/\text{hr} = \mathbf{\$10/day} \quad \text{(down from \$160!)}$$

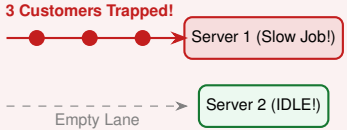
- **New Total Daily Cost (2 Mechanics):**

$$(2 \times \$88) + \$10 = \$176 + \$10 = \mathbf{\$186/day}$$

- **Net Bottom-Line Savings:**
$\$248 - \$186 = \mathbf{\$62/day\ saved}$ while slashing wait from 40m → 2.5m!

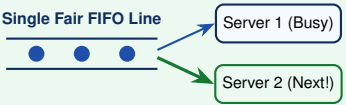
Why Pooling Works: Single Serpentine Line ($M/M/S$) vs. Separate Lines

Design 1: Separate Lines (S Independent $M/M/1$ Queues)



- **The Failure Mode:** One complex customer at Server 1 traps everyone behind them while Server 2 sits idle!
- Triggers jockeying and "wrong-line rage."

Design 2: Single Pooled Snake Line ($M/M/S$)



- **Mathematical Guarantee:** No server is *ever* idle while a customer waits!
- Cuts average wait W_q by **50%–80%** with the exact same headcount (Wendy's, Trader Joe's, TSA, Banks).

Staffing Multi-Server Centers with Waiting-Line Tables (Bank Case)

Excerpt of Poisson Queue Table D.5 (L_q by λ/μ and M)

Ratio λ/μ	**M = 1**	**M = 2**	**M = 3**	**M = 4**	**M = 5**
0.50	0.5000	0.0333	0.0030	0.0002	0.0000
0.67	1.3467	0.0844	0.0084	0.0008	0.0001
0.80	3.2000	0.1524	0.0189	0.0021	0.0002
0.90	**8.1000**	**0.2285**	**0.0300**	**0.0037**	0.0004
1.50	*Unstable*	1.9286	0.2368	0.0450	0.0079
2.50	*Unstable*	*Unstable*	3.5112	0.5331	0.1297

• Notice that λ/μ is the **traffic intensity** (minimum number of servers needed for stability is $M > \lambda/\mu$).

Bank Teller Staffing ($\lambda = 18$/hr, $\mu = 20$/hr)

Traffic ratio: $\lambda/\mu = 18/20 = \mathbf{0.90}$. Compare $M = 1, 2, 3$ tellers using $W_q = L_q/\lambda$:

- $M = 1$ **Teller** ($\rho = 90\%$)**:** $L_q = 8.10$ cust $\implies W_q = \frac{8.10}{18}$ hr $=$ **27.0 minutes**!
- $M = 2$ **Tellers** ($\rho = 45\%$)**:** $L_q = 0.2285$ cust $\implies W_q = \frac{0.2285}{18}$ hr $=$ **45.7 seconds**!
- $M = 3$ **Tellers** ($\rho = 30\%$)**:** $L_q = 0.0300$ cust $\implies W_q = \frac{0.0300}{18}$ hr $=$ **6.0 seconds**!

Model C ($M/D/1$): Process Automation Economics (Garcia-Golding)

The Capital Investment Decision

- **Current Manual Operation:**
  - Recycling trucks arrive at $\lambda = $ **8 trucks/hr**.
  - Drivers currently wait an average of $W_q^{\text{old}} = $ **15 min** (0.25 hr) before unloading.
  - Truck & driver cost $= $ **\$60/hr** $\implies$ **\$15.00 wait cost/trip**.
- **Proposed Automated Compactor ($M/D/1$):**
  - Processes trucks in a **constant** 5 **min** ($\mu = $ **12 trucks/hr**, $\sigma_s = 0$).
  - Amortized equipment cost $= $ **\$3.00/truck unloaded**.

$M/D/1$ Engineering ROI Analysis

- **1. New Constant-Service Queue Wait (W_q):**

$$W_q^{(M/D/1)} = \frac{\lambda}{2\mu(\mu - \lambda)} = \frac{8}{96} \text{ hr} = \textbf{5.0 min}$$

- **2. New Waiting Cost per Trip (saves \$10.00/trip):**

$$\frac{1}{12} \text{ hr} \times \$60/\text{hr} = \textbf{\$5.00/trip} \quad \text{(vs. \$15.00 manual)}$$

- **3. Net Savings After Compactor Cost (\$3/trip):**

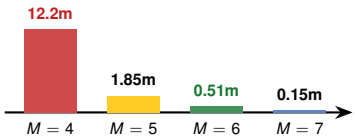
\$10.00 wait savings $-$ \$3.00 $=$ **\$7.00 net profit/trip!**

Gemini Visual Intuition 5: Pooled Snake Line & SLA Optimizer

Copy-Paste Prompt for Gemini AI (Replaces Table D.5)

```
A hospital phlebotomy lab has Poisson arrivals lambda = 44/hr.
Each phlebotomist serves mu = 12/hr (5 min) at $35/hr wage;
patient wait costs $50/hr.  For M = 4, 5, 6, 7 pooled servers:
1) Plot a bar chart of wait Wq (min) and total hourly cost.
2) Find minimum M for SLA Wq <= 3 min and cost-optimal M.
```

Nonlinear Collapse of Wait W_q as Servers M Rise:



Instant $M/M/S$ Optimization Table ($\lambda/\mu = 3.67$)

Staff M	Util ρ	Wait W_q	Staff Cost	Total Cost/Hr
$M = 4$	91.7%	12.2 min	\$140	\$587/hr
M = 5	**73.3%**	**1.85** min	**\$175**	**\$243**/hr
M = 6	**61.1%**	**0.51** min	**\$210**	\$**229**/hr (Min!)
$M = 7$	52.4%	0.15 min	\$245	\$251/hr

- **SLA Threshold ($W_q \leq$ 3m):** $M = 5$ servers ($W_q = 1.85$m).
- **Cost-Optimal Staffing:** $M = 6$ servers (cuts wait to 31 sec and saves \$358/hr vs. running lean at $M = 4$!).

FORMULA REMINDER: Minimum Servers: $M > \frac{\lambda}{\mu}$ | $\rho = \frac{\lambda}{M\mu} < 100\%$ | Hourly Total Cost $= M \cdot C_{\text{server}} + \lambda W_q \cdot C_{\text{wait}} = M \cdot C_{\text{server}} + L_q \cdot C_{\text{wait}}$

Beyond the Math: The Psychology of Waiting Lines (WSJ & Maister)

David Maister's 4 Laws of Perceived Wait

1. **Occupied Time Feels Shorter Than Unoccupied Time:**
   - High-rise elevator mirrors, restaurant menus in line, Disney interactive queue games, baggage-claim walk distance.
2. **Pre-Process Waits Feel Longer Than In-Process Waits:**
   - Nurse taking vitals immediately in an exam room (or chips & salsa at a table) converts "waiting for service" into "service has begun!"

Anxiety, Uncertainty & Queue Fairness (WSJ)

3. **Uncertain / Unexplained Waits Feel Longer:**
   - Posting conservative wait estimates (*"Wait from this point: 15 min"* when actual is 10 min) creates delight via under-promising and over-delivering.
4. **Unfair Waits Trigger "Queue Rage":**
   - In WSJ's *Science of Standing in Line*, watching a later arrival get served first in an adjacent lane is the #1 driver of customer rage—making **single serpentine lines** a dual mathematical *and* psychological triumph!

Strategic Queue Architecture: When to Pool vs. When to Segment

Rule 1: Pool Homogeneous Customers ($M/M/S$)

- When customers have **similar service-time distributions**, always merge separate lanes into a **single serpentine queue**.
- Pooling prevents a single slow transaction from starving a server while another server sits idle.
- **Wait-Time Reduction:** Typically slashes W_q by 50%–80% at zero labor cost.

Rule 2: Segment High-Variance vs. Low-Variance!

- **Wait—is full pooling ALWAYS best?** Recall Scenario 3 and the WSJ *TSA Security Lines* article!
- If you pool **low-variance fast customers** (TSA PreCheck: shoes/belts on, laptops in bag, $\sigma \approx 0$, or RERUNs) into the same line as **high-variance slow customers** (families with strollers, liquids, $\sigma =$ high, or RUNs), the high variance **contaminates** every fast customer!
- **Optimal Architecture:** Separate **Low-σ Express / PreCheck** from **High-σ Standard**, then use a **pooled snake line within each segment**!

Synthesis Rule: **Pool across servers** to share capacity, but **segment across job types** when service-time variance (σ_s) or mean ($1/\mu$) differs dramatically!

Interactive Mini-Case: UrgentCare Clinic Triage Redesign

The Operational Dilemma (2 Doctors on Duty)

- **Total Demand:** $\lambda_{\text{total}} = 6$ **patients/hr**:
  - 50% ($\lambda_F = 3$/hr) are **Fast-Track** (strep/flu swabs: standardized protocol, $\sigma_s = 0$).
  - 50% ($\lambda_C = 3$/hr) are **Complex** (chest pain, lacerations: Exponential $\sigma_s = 15$m).
- **Capacity:** $M = 2$ doctors, each serving $\mu = 4$ **patients/hr** (15 min mean). Utilization $\rho = \frac{6}{2\times 4} = 75\%$.

Compare 3 Queue Architectures (Find W_q in Min)

1. **Option A (2 Separate Mixed $M/M/1$ Lines):** Each doctor has their own unpooled line ($\lambda = 3$/hr, $\mu = 4$/hr, high variance). What is $W_q^{(A)}$?
2. **Option B (1 Pooled $M/M/2$ Serpentine Line):** All $\lambda = 6$/hr patients join 1 snake line for $M = 2$ doctors ($\mu = 4$/hr). From Table D.5 at $\lambda/\mu = 1.50$, $M = 2$, $\mathbf{L_q = 1.9286}$. What is $W_q^{(B)}$?
3. **Option C (Segmented Triage):** Dedicate Doctor 1 to **Constant-Service** $M/D/1$ **Fast-Track** ($\lambda_F = 3$, $\mu = 4$) and Doctor 2 to **Complex** $M/M/1$ ($\lambda_C = 3$, $\mu = 4$). What are $W_q^{(\text{Fast})}$ and $W_q^{(\text{Complex})}$?

FORMULA REMINDER: $M/M/1$: $W_q = \frac{\lambda}{\mu(\mu-\lambda)}$ | $M/M/2$: $W_q = \frac{L_q}{\lambda}$ (with $L_q = 1.9286$ at $\lambda/\mu = 1.5$) | $M/D/1$: $W_q = \frac{\lambda}{2\mu(\mu-\lambda)}$

The Executive Queueing Playbook: 4 Zero-CapEx Levers

Lever 1: Slash WIP & Balance Shifting Bottlenecks

- **Physics:** $\text{TAT} = \text{WIP} \times \text{CT}_{\text{BN}}$ (Little's Law).
- **Action:** Cap WIP release into the system and use cross-trained floaters ($t_i = x_i/(1+\alpha_i)$) or complementary product mixes (αRUN $+ (1-\alpha)$RAP) so no station approaches $\rho = 100\%$.

Lever 2: Engineer Out Service Variability ($M/M/1 \to M/D/1$)

- **Physics:** $W_q \propto \frac{\rho}{1-\rho}(\text{CV}_a^2 + \text{CV}_s^2)$.
- **Action:** Standardize checklists, pre-screen documents, use appointment scheduling ($\text{CV}_a \to 0$), and automate routine steps ($\text{CV}_s \to 0$) to cut W_q by 50%+ at the same utilization.

Lever 3: Pool Parallel Servers & Segment Outliers

- **Physics:** Single-queue $M/M/S$ eliminates idle servers while customers wait in adjacent lanes.
- **Action:** Replace separate lines with a single serpentine queue, while separating ultra-fast/low-σ transactions (TSA PreCheck / Express) from complex high-σ cases.

Lever 4: Redesign the Perceived Wait (Maister's Laws)

- **Psychology:** Subjective wait depends on anxiety, fairness, and idle attention.
- **Action:** Start intake immediately (pre-process $\to$ in-process), occupy attention, guarantee FIFO fairness, and post slightly conservative wait times.

Session 7 Executive Summary: 5 Enduring Rules of Service Utilization

Core Principle	Mathematical Law	Managerial Rule of Thumb
1. Little's Law	$L_s = \lambda W_s, \quad L_q = \lambda W_q$ Average TAT = Average WIP $\times$ $\mathrm{CT_{BN}}$	Never flood a facility with excess WIP; choking the floor linearly inflates customer turnaround time.
2. The Stability Cliff	$\rho = \dfrac{\lambda}{M\mu} = \dfrac{\mathrm{CT_{BN}}}{\mathrm{IAT}} < 100\%$	At $\rho \geq 100\%$, queues grow to infinity regardless of how low variability is (even at $\sigma = 1\mathrm{m}$).
3. The Hockey-Stick Curve	$W_q \propto \left(\dfrac{\rho}{1-\rho}\right)\left(\dfrac{\mathrm{CV}_a^2 + \mathrm{CV}_s^2}{2}\right)$	High-variance services ($\sigma = 14\mathrm{m}$) require 20%–25% capacity buffers ($\rho < 75\%$–80%); 99.5% utilization is a trap!
4. Shifting Bottlenecks	$\mathrm{CT}_j(\alpha) = \alpha t_{A,j} + (1-\alpha) t_{B,j} < \mathrm{IAT}$	Balance workloads across stations using cross-trained floaters or complementary product mixes (RUN + RAP).
5. Pooling & Psychology	$W_q^{(M/M/S)} \ll W_q^{(M/M/1)}$ $W_q^{(M/D/1)} = \frac{1}{2} W_q^{(M/M/1)}$	Pool parallel servers into a single fair serpentine line, standardize service times, and occupy customer wait time.

In-Class Graded Simulation Quiz: Submit All 13 HBP Questions

Scenarios 1 & 2 (Questions 1–9)

- **Q1 (Scen. 1):** At Initial WIP $= 40$, what is the Red Policy TAT (in hours, nearest integer)?
- **Q2 (Scen. 1):** Based on your WIP sweep (120, 200, 280), what is Red Policy TAT at WIP $= 240$ (in hours)?
- **Q3 (Scen. 1):** Steady-state 15-step process, WIP $= 720$, bottleneck task time $= 5.0$ min. What is Average TAT (in hours)?
- **Q4 (Scen. 2):** Which workstation is the initial bottleneck before allocating the 5th worker? (*a: WS1, b: WS2, c: WS3, d: WS4*)
- **Q5 (Scen. 2):** With the entire 5th worker on the initial bottleneck (2.0 workers), what is its new task time (in min)?
- **Q6 (Scen. 2):** After assigning the 5th worker to the initial bottleneck, which station becomes the new bottleneck?
- **Q7 (Scen. 2):** If you split the 5th worker 50%/50% (1.5 each) on the two slowest stations, can the system meet demand? (*a: No, b: Yes*)
- **Q8 (Scen. 2):** Lowest possible cycle time if all 4 stations are balanced (nearest 0.1 min)?
- **Q9 (Scen. 2):** Highest 12-week total revenue achieved in Scenario 2 (nearest \$1, 000)?

Scenarios 3 & 4 (Questions 10–13)

- **Q10 (Scen. 3):** Holding mean task time at 18 min, as task-time variability (σ) increases across policy types, Average TAT: (*a: increases, b: decreases, c: stays roughly the same*)
- **Q11 (Scen. 3):** For 100% RUNs ($\sigma = 14$m), as mean task time increases ($18 \rightarrow 19 \rightarrow 19.9$ min), Average TAT: (*a: increases, b: decreases, c: stays roughly the same*)
- **Q12 (Scen. 3):** For 100% RUNs, which task-time increase causes the **larger** jump in Average TAT? (*a: 18 to 19 min, b: 19 to 19.9 min*)
- **Q13 (Scen. 4):** What is the highest 12-week total revenue you can achieve in Scenario 4 by optimizing the RUN / RERUN / RAP product mix (rounded to nearest \$1, 000)?

FORMULA REMINDER: $\mathrm{TAT} = \mathrm{WIP} \times \mathrm{CT_{BN}} \mid t_i = \dfrac{x_i}{1+\alpha_i} \mid \mathrm{CT_{ideal}} = \dfrac{\sum_{i=1}^{4} x_i}{5} \mid \rho = \dfrac{\mathrm{CT_{BN}}}{\mathrm{IAT}} < 1 \mid \mathrm{CT}_j(\alpha) = \alpha t_{\mathrm{RUN},j} + (1-\alpha) t_{\mathrm{RAP},j} < \mathrm{IAT}$